计算机体系结构

10. 指令级并行性 I

李建华
计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburg, and eth. The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

本讲提纲

- **指令级并行的概念**
- 循环展开和指令调度
- 指令的动态调度
- 分支预测技术
- 多指令流出技术

ILP相关概念

- 指令流水线中指令之间潜在的并行性（重叠处理）称为指令级并行 (Instruction-Level Parallelism)。
- 接下来详细介绍 ILP：
  - 与ILP相关的一些基本概念
  - 有效提升ILP的硬件和软件技术:
    - 硬件和软件互相配合，才能最大限度地 挖掘出 程序中存在的指令级并行。
  - 感兴趣的同学可阅读：

DEC Western Digital Laboratory

Limits of Instruction-Level Parallelism

David W. Wall

ILP相关概念（绪）

- 流水线处理器的实际CPI：
  - 理想流水线的CPI加上各类停顿的开销：

$$CPI_{流水线} = CPI_{理想} + 停顿_{结构冲突} + 停顿_{数据冲突} + 停顿_{控制冲突}$$

  - 理想CPI是衡量流水线最高性能的一个指标
  - CPI：Cycles Per Instruction
  - IPC：Instructions Per Cycle

倒数关系

ILP相关概念（续）

- 循环级并行：使一个循环中的不同循环体并行执行。
  - 优化循环是最常见、最基本的提升ILP的方法
  - 是指令级并行研究的重点**（程序中80%以上时间用于循环的处理）**
  - 例如，考虑下述循环代码段：

```
for (i=1; i<=500; i++)
    a[i]=a[i] + s;
```

    - 每一次循环都可以与其他的循环重叠并行执行；
    - 在每一次循环的内部，却没有任何的指令级并行性。

ILP相关概念（绪）

- 相关与流水线冒险之间的关系 [Review]
  - 流水线冒险是指：对于具体的流水线来说，**相关**导致指令流中的下一条指令不能在指定的时钟周期**执行**。
  - 相关是程序固有的一种属性，它反映了程序中指令之间的相互依赖关系。
  - 具体的一次相关是否会导致流水线**冒险**的发生以及该冒险的解决需要多长时间的停顿，则需要结合流水线的属性来分析。
    - 可以回顾上一个topic的内容

ILP相关概念（绪）

- 程序顺序（Program Order）：由源程序确定的在完全**串行方式**下指令的执行顺序。
  - 此前所讲的内容里面：由于相关造成的冒险可能影响程序结果的正确性，因此指令是按照**程序顺序**处理的。
- 实际上，正确地执行程序（结果正确），必须保持的
  - 序，都
    - 原来程序中异常是怎么发生的，**改变执行顺序**后还是怎么发生。
    - 为了提升性能，可以弱化为：**指令执行顺序的改变不能导致程序中发生新的异常。**

换句话来说，就是程序顺序可以改变！为什么要改变程序顺序呢？

保持数据相关

- **如果能保持程序的数据相关和控制相关，就能保持程序的数据流和异常行为。**
  - 从而确保最终指令处理的结果是正确的
- 示例:

```
DADDU   R2, R3, R4
BEQZ    R2, L1
LW      R1, 0(R2)
L1:
```

必须保持对R2的数据相关，否则可能会破坏数据流，从而影响计算结果的正确性！

  - **问题**：In-order execution时，如何保持数据流?
  - Out-of-order execution情况下呢?

保持控制相关

- 数据流：指**数据值**从其**产生者指令**到其**消费者指令**的实际**流动**。【如：数据前推也要保持数据流！】
  - 然而，分支指令使得数据流具有动态性，因为它使得某条指令的源操作数可能有多个来源。
  - 所以，仅仅保持数据相关性是不够的，还须保持控制相关，才能够保持数据流。
- 示例：

```
      DADDU   R1, R2, R3
      BEQZ    R4, L1
      DSUBU   R1, R5, R6
L1 :  …
      OR      R7, R1, R8
```



保持控制相关（续）

- 特殊场景：有时不遵守控制相关既不影响异常行为，也不改变数据流。
  - 这时候可以大胆地进行指令调度，把失败分支中的指令调度到分支指令之前。
  - 示例：

```
        DADDU     R1，R2，R3
        BEQZ      R12，Skipnext
        DSUBU     R4，R5，R6
        DADDU     R5，R4，R9
skip:   OR        R7，R8，R9
```

  - 假设 **skip后 R4不再使用**，且**DSUBU不产生异常**，则可以将DSUBU调度到分支之前。（OoO）

ILP 总结

- ILP相关的基本概念，到这里就结束了。
  - 很抽象，难以理解；
  - 请各位花时间思考这些概念

- 这不够，同学们需要**认真阅读教材中的相关内容。**
  - 教材的论述逻辑更加清晰，内容更加具体、详实。
  - 你甚至要追根溯源，去尝试检索和阅读原始文献。

本节提纲

- 指令级并行的概念
- **循环展开和指令调度**
  - 提升ILP的软件方法
- 指令的动态调度
- 分支预测技术
- 多指令流出技术

循环展开与指令调度

- 这二种技术的组合可充分开发ILP，通过构建不相关的指令序列，让流水线高效地稳定地工作。
- Loop Unrolling & Instruction Scheduling是提升ILP最简单和最常用的方法:
  - 开发循环级并行性 — 循环的不同迭代之间存在的并行性。
  - 核心原理：把循环展开后，通过**寄存器重命名**和**指令调度**来开发更多的ILP。

接下来，我们通过一个示例介绍这两项技术。

指令调度

- 编译器完成指令调度的能力受限于两个特性：
  - 程序固有的指令级并行性；
  - 流水线功能部件的延迟。
- **示例：**

❶ 假设浮点流水线延迟如下：

产生结果的指令	**使用结果**的指令	延迟（时钟周期数）
浮点计算	另一个浮点计算	**3**
浮点计算	浮点store（S.D）	**2**
浮点load（L.D）	浮点计算	**1**
浮点load（L.D）	浮点store（S.D）	**0**

指令调度示例

❷ 假设采用MIPS的5段流水线：

- 分支延迟：1个时钟周期；【请看COD-5寻找答案】
- **注意：**分支的条件判断和目标计算在ID进行；
- 整数运算部件是全流水或者整数运算部件数量充足
  - 也就是假定没有资源冲突

指令调度示例

❸ 对于下面的源代码，转换成MIPS汇编语言，在不进行指令调度和进行指令调度两种情况下，分析其代码一次循环所需的执行时间。

```
for (i=1; i<=1000; i++)
    x[i] = x[i] + s;
```

解： 把该程序翻译成MIPS汇编语言代码：

假设：

① **R1的初值**是指向**第一个元素**

② **8（R2）**指向**最后一个元素**

③ 浮点寄存器F2：用于保存常数s

MARS!
LARS!
RARS!

Try to write MIPS assembly code for this function!

指令调度示例

```
Loop:  L.D      F0, 0（R1）
       ADD.D    F4, F0, F2
       S.D      F4, 0（R1）
       DADDIU   R1, R1, #-8
       BNE      R1, R2, Loop
```

其中：

- 整数寄存器R1指向向量中的当前元素
- R1初值为向量中最高端元素的地址

指令调度示例

- 不进行指令调度的情况下，程序的实际执行情况：

	指令流出时钟
Loop: L.D F0, 0(R1)	1
（空转）	2
ADD.D F4, F0, F2	3
（空转）	4
（空转）	5
S.D F4, 0(R1)	6
DADDIU R1, R1, # -8	7
（空转）	8
BNE R1, R2, Loop	9
（空转）	10

数据前推在此处没有作用！

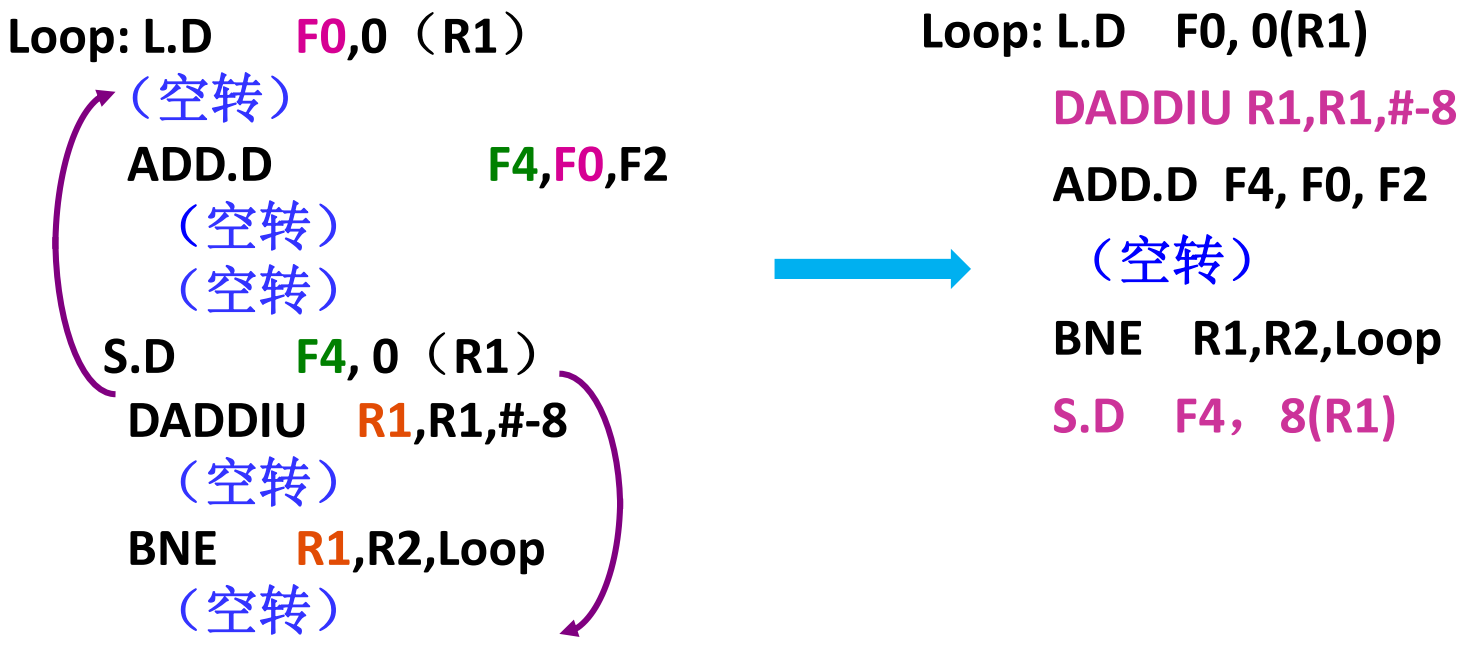
– 每个元素的操作需要10个时钟周期，其中5个是空转周期。

指令调度示例

- 如果进行如下的指令调度：
  - 把DADDIU指令调度到L.D指令和ADD.D指令之间的“空转”拍；
  - 把S.D指令放到了分支指令的延迟槽中；
  - 对存储器地址偏移量进行调整；

	指令流出时钟
Loop: L.D F0, 0(R1)	1
（空转）	2
ADD.D F4, F0, F2	3
（空转）	4
（空转）	5
S.D F4, 0(R1)	6
DADDIU R1, R1, # -8	7
（空转）	8
BNE R1, R2, Loop	9
（空转）	10

指令调度示例


Loop: L.D F0,0（R1）
（空转）
ADD.D F4,F0,F2
（空转）
（空转）
S.D F4, 0（R1）
DADDIU R1,R1,#-8
（空转）
BNE R1,R2,Loop
（空转）
Loop: L.D F0, 0(R1)
DADDIU R1,R1,#-8
ADD.D F4, F0, F2
（空转）
BNE R1,R2,Loop
S.D F4，8(R1)

指令调度示例

	指令流出时钟
Loop: L.D F0, 0(R1)	1
DADDIU R1, R1, #-8	2
ADD.D F4, F0, F2	3
(空转)	4
BNE R1, R2, Loop	5
S.D F4, 8(R1)	6

- 一轮循环的操作时间从10个时钟周期减少到6个，其中5个周期是有指令执行的，1个为空转周期。

指令调度示例

- 示例的问题及解决方案
  - **问题：**只有L.D、ADD.D和S.D这3条指令是真正有用操作。
    - 占用3个时钟周期
    - 而DADDIU、空转和BNE这3个时钟周期都是附加的循环控制开销。（**开销是怎么来的呢**？）
    - 能否进一步优化?
  - **优化方案：**循环展开技术
    - 把循环体的代码复制多次并按顺序排列，然后相应调整循环的结束条件。
    - 这给编译器进行指令调度带来了更大的空间。

循环展开示例

- **循环展开示例：**将上述例子中的循环展开4次得到4个循环体，然后对展开后的指令序列在不调度和调度两种情况下，分析代码的性能。假定R1的初值为32的倍数，即循环次数为4的倍数。消除冗余的指令，并且不要重复使用寄存器。

解：无需在循环体后面增加补偿代码，分配寄存器（不重复使用寄存器）如下：

- F0、F4：用于展开后的第1个循环体
- F2：保存常数s
- F6、F8：第2个循环体
- F10、F12：第3个循环体
- F14、F16：第4个循环体

- 展开后没有调度的代码如下：

			指令流出时钟
Loop:	L.D	F0,0(R1)	1
	（空转）		2
	ADD.D	F4,F0,F2	3
	（空转）		4
	（空转）		5
	S.D	F4, 0（R1）	6
	L.D	F6,-8(R1)	7
	（空转）		8
	ADD.D	F8,F6,F2	9
	（空转）		10
	（空转）		11
	S.D	F8, -8（R1）	12
	L.D	F10,-16(R1)	13
	（空转）		14
	ADD.D	F12,F10,F2	15
	（空转）		16
	（空转）		17
	S.D	F12,-16(R1)	18
	L.D	F14,-24(R1)	19
	（空转）		20
	ADD.D	F16,F14,F2	21
	（空转）		22
	（空转）		23
	S.D	F16,-24(R1)	24
	DADDIU	R1,R1,#-32	25
	（空转）		26
	BNE	R1,R2,Loop	27
	（空转）		28

循环展开示例

- 性能分析:
  - 这个循环每遍共使用了28个时钟周期。
  - 有4个循环体，完成4个元素的操作。平均每个元素使用28/4=7个时钟周期
  - 原始循环的每个元素需要10个时钟周期。节省的时间：从减少循环控制的开销中获得的。
  - 在整个展开后的循环中，实际指令只有14条，其他14个周期都是空转。

总的来说，这样并不高效。

循环展开+指令调度

- **综合示例：**对指令序列进行优化调度，以减少空转周期：

			指令流出时钟
Loop:	L.D	F0,0(R1)	1
	L.D	F6,-8(R1)	2
	L.D	F10,-16(R1)	3
	L.D	F14,-24(R1)	4
	ADD.D	F4,F0,F2	5
	ADD.D	F8,F6,F2	6
	ADD.D	F12,F10,F2	7
	ADD.D	F16,F14,F2	8
	S.D	F4,0(R1)	9
	S.D	F8,-8(R1)	10
	DADDIU	R1,R1,#-32	12
	S.D	F12,16(R1)	11
	BNE	R1,R2,Loop	13
	S.D	F16,8(R1)	14

循环展开+指令调度

- 性能分析：
  - 没有数据相关引起的空转等待。
    - 整个循环仅仅使用了14个时钟周期。
    - 平均每个元素的操作使用14/4=3.5个时钟周期。
  - 结论：通过循环展开、寄存器重命名和指令调度，可以有效地开发出指令级并行。

循环展开+指令调度

- 循环展开和指令调度时要注意以下几个方面：
  - 保证正确性
    - 在循环展开和调度过程中尤其要注意两个地方的正确性：**循环控制**和**操作数偏移量的修改。**
  - 注意有效性
    - 只有能够找到不同循环体之间的无关性，才能有效地使用循环展开。
  - 使用不同的寄存器，否则可能导致新的冲突；
  - 注意对存储器数据的相关性分析 （我们不讨论）
  - 注意新的相关性
    - 由于原来的循环中不同轮次的迭代在展开后都到了同一次循环体中，因此**可能带来新的相关性。**

Next Topic: ILP 2

3.1	Instruction-Level Parallelism: Concepts and Challenges	148
3.2	Basic Compiler Techniques for Exposing ILP	156
3.3	Reducing Branch Costs with Advanced Branch Prediction	162
3.4	Overcoming Data Hazards with Dynamic Scheduling	167
3.5	Dynamic Scheduling: Examples and the Algorithm	176
3.6	Hardware-Based Speculation	183
3.7	Exploiting ILP Using Multiple Issue and Static Scheduling	192
3.8	Exploiting ILP Using Dynamic Scheduling, Multiple Issue, and Speculation	197
3.9	Advanced Techniques for Instruction Delivery and Speculation	202